

Day 58

* File or streams in python

Index

=> Purpose of files.

=> Types of Application.

a) Non-persistent Applications.

b) Persistent Applications.

=> Definition of file.

=> Definition of stream.

=> Operations on files.

a) write operation

b) read operation

=> Types of files.

a) Text files

b) Binary files.

=> file opening modes.

1) r 2) w 3) a 4) r+ 5) w+ 6) a+

7) x 8) x+

=> Number of approaches to open file.

a) By using open()

b) By using "with open() as".

=> programming Ex.

=> pickling (Object serialization) and un-pickling (Object De-serialization)

=> Implementation of pickling & unpickling operations.

=> pickle module.

=> programming Example.

=> working with CSV files and operations.

=> CSV module

=> Implementation of CSV operations with CSV module

a) csv.reader

b) csv.writer

c) csv.DictReader

d) csv.DictWriter

=> Ex.

=> working on OS Based operations.

=> OS module

=> implementation OS Based operations with OS module

* Types of Application in File:-

=> The purpose of files in any programming language is that "to maintain data persistency".

=> The process of storing the data permanently is called "Data persistency".

=> In this context, we can develop two types of Applications.

They are:

(1) Non-persistent Applications

(2) Persistent Application.

=> In Nonpersistent Application development, we read the data from keyboard, stored in main memory (RAM) in the form of objects, processed and whose results displayed on monitor.

example) All our previous examples comes under non-persistent Appn.

=> we know that Data stored in main memory (RAM) is temporary.

=> In persistent Applications development, we read the data from keyboard, stored in main memory (RAM) in the form of objects, processed and whose results stored permanently

=> in industry, we have two ways store the Data permanently

They are:

1) By using Files

2) By using RDBMS Database softwares
(Oracle, MySQL, MongoDB, DB2,
SQL server, etc.).

* Data persistency by using Files of Python:-

Def. of File:

=> A file is a collection of Records.

=> Files Resides in secondary memory (HDD)

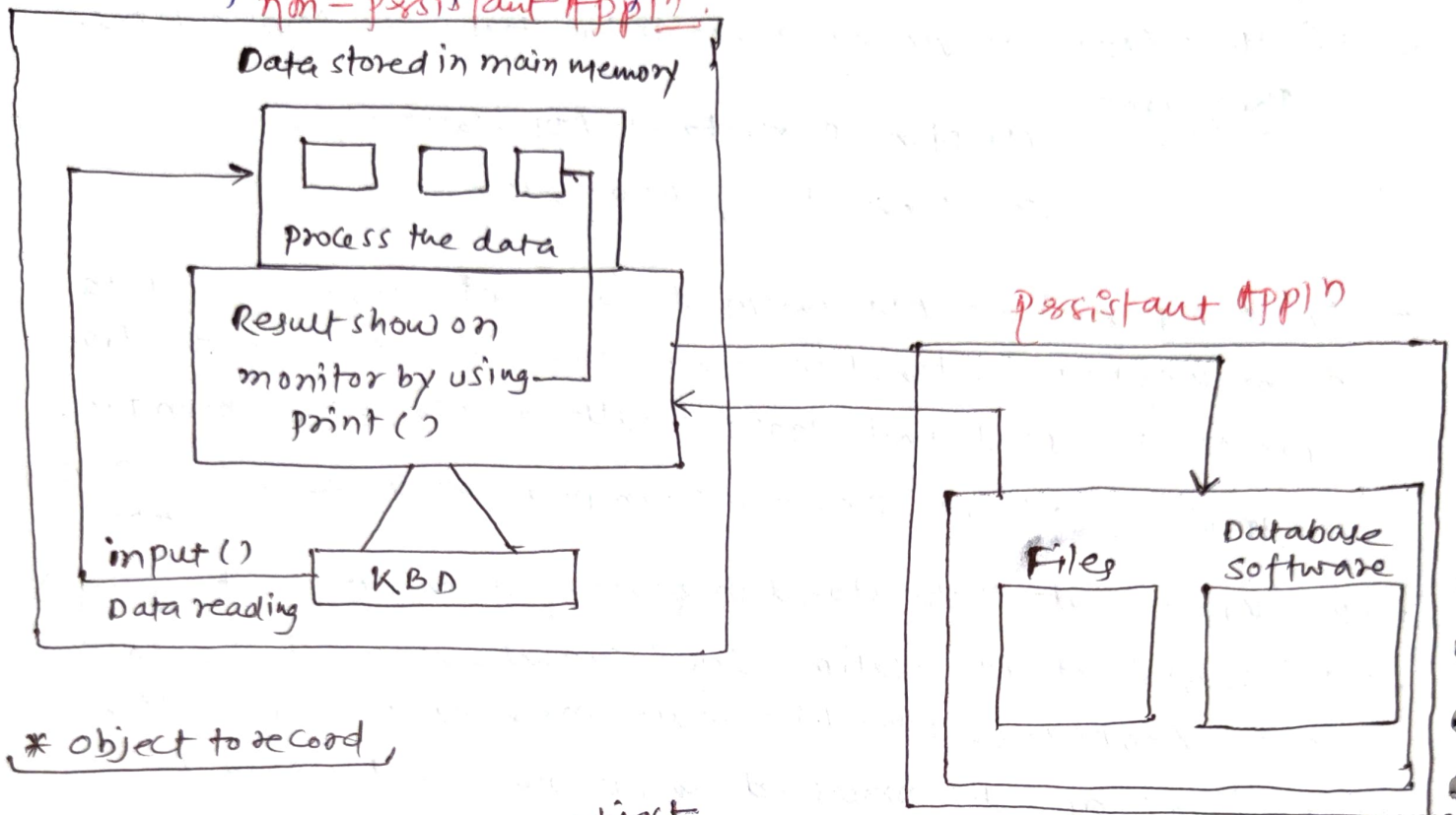
=> Technically, file name is named locations in secondary memory

=> the purpose of files is that "To get Data persistency".

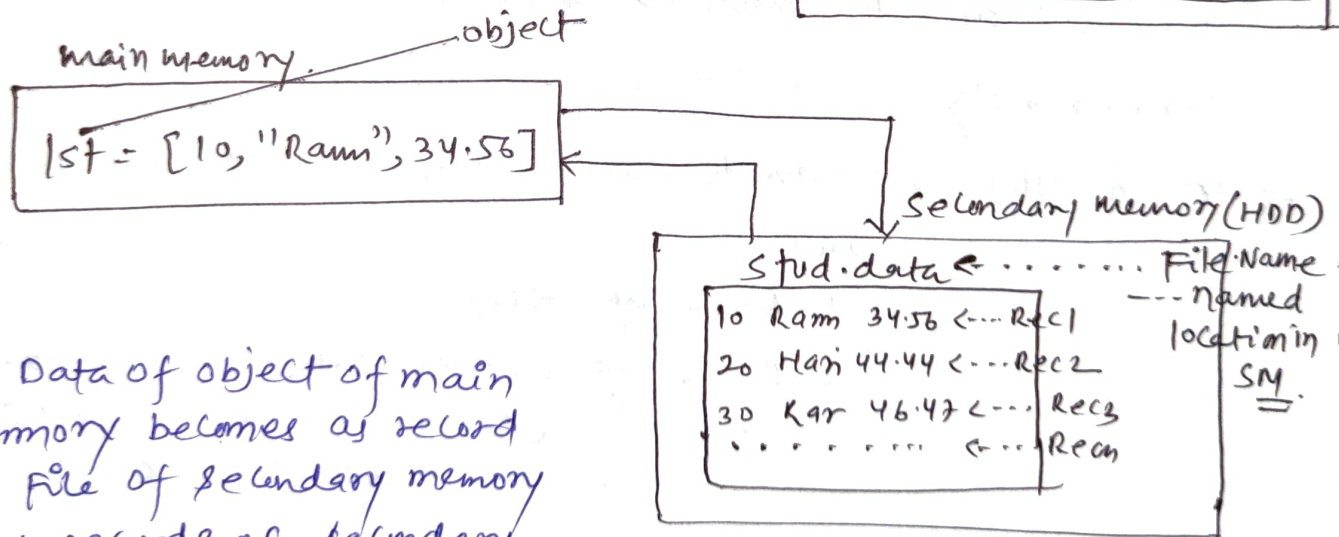
⇒ All the objects data of main memory become records in file of secondary memory becomes the objects in main memory.

Def of stream:

⇒ The flow of data between object(s) of main memory and files of secondary memory is called "stream".



* object to record



- The Data of object of main memory becomes as record in file of secondary memory and records of secondary memory become as object in main memory - stream

* Operations on files :-

→ on the files, we can perform two types of operations. They are :-

- ① write operation.
- ② Read operation.

① write operation :-

→ The purpose of write operation is that "to transfer or save the object data of main memory as record in the file of secondary memory".

Steps :-

- 1) Choose the file Name.
- 2) open the file Name in write Mode.
- 3) perform cycle of write operations.

⇒ while we are performing write operations, we get the following expectations.

- a) IOError
- b) OSError
- c) FileNotFoundError.

② Read operation :-

→ The purpose of read operation is that "to transfer or read the record from file of secondary memory into the object of main memory".

Steps :-

- 1) Choose the file Name.
- 2) open the file Name in Read Mode.
- 3) perform cycle of read operations.

⇒ while we performing read operations, we get the following exceptions.

- a) FileNotFoundError
- b) EOFError

* Types of Files in python :-

→ In python programming, we have two types of Files.

They are:

- ① Text Files
- ② Binary Files.

① Text Files.

→ Text Files are those which are containing Alphabets, Digits, and symbols.

→ Text Files are denoted by a letter 't'.

→ By default every file treated as Text File.

→ example: .c, .cpp, .txt, .docx, .xlsx, .py etc.

② Binary Files

→ Binary Files are those which are containing the data in the form of Binary.

→ Binary Files are denoted by a letter 'b'.

→ example: All types of Image Files (.Jpeg, .gif, .Jpg, .png) all types of audio, video files

Day (59):-

* file opening modes in python:-

→ The purpose of File Opening Modes in python is that "in which mode we are opening the file".

⇒ In python programming, we have 8 File opening modes.
They are:-

① r:-

→ This mode is used for opening the file in READ mode and performs read operation only.

→ If we open the file name in "r" mode and if that file does not exist then we get FileNotFound Error.

→ The default file mode is "r".

② w:-

→ This mode is used for creating a new file and opening that file in write mode and we can perform write operations.

→ If we choose new file then file created and opened in write mode and we can perform write operations.

→ If we choose existing file then that file opened in WRITE mode and existing data overlapped with new data.

③ a:-

→ w के साथ same hai, Bas, Data APPENDED with new data.

④ r+:-

→ This mode is used for opening the file in Read mode.

→ If we open the file name in "r+" mode and if that file does not exist then we get FileNotFound Error.

→ If we open the file name in "r+" mode then the file opened in read mode. First performs Read operations and later performs write operation also.

⑤ w+ :-

- This mode is used for creating a new file and opening that file in write mode.
- if we choose New File then that file created and opened in write mode and we can perform write operations.
- if we choose Existing File then that file opened in write mode and existing data overlapped with new data.
- if we open filename (New File or Existing File) in "w+" then first we can perform write operation and later we can also perform read operation.

⑥ a+ :-

- w+ ke jaisa same hai, Bas Existing Data Appended with new data

⑦ x :-

- This mode is used for creating a New File and opening the File Name exclusively in write mode and we can perform write operation.
- if we open the existing file name in "x" mode then we get file exist error.

⑧ x+ :-

- This mode is used for creating a New File and opening the File Name exclusively in write mode and first we can perform write operations and later we can also perform read operation.
- if we open the existing file name in "x+" mode then we get "file exist error"

* Number of approaches to open the files in python

→ TO perform any type of operation on files, we must open the file by using two approaches.

They are:

- 1) By using `open()`
- 2) By using `"with open() as"`.

1) By using open()

Syntax:- `varname = open("File Name", "File Mode")`,

- ⇒ "varname" represents an object pointing the file and it is called "File pointer" and whose type is `<class, _io.TextIOWrapper>`
- ⇒ `open()` is a pre-defined function present in built-in module and it is used for opening the filename in specified file mode.
- ⇒ "FileName" represents Name of the file.
- ⇒ "File Mode" represents `r, w, a, r+, w+, a+, x, x+`
- ⇒ Once we use the file name with `open()` then it is highly recommended to close the file by using `close()` for maintaining consistency of file data (manual closing of the file)
- ⇒ Hence `open()` does not provide auto-closeability of file.

2) By using "with open() as"

Syntax: `with open("File Name", "File Mode") as varname:`

Block of statement

- performs file operations

Other statements in program

=> "with" and "as" are the keywords.

=> open() is predefined function in built-in module and it is used for opening the filename in specified file mode.

=> "filename" represents name of the file

=> "filemode" represents r, w, a+, wt, at, x, xt

=> The execution process of "with open(---) as" is that "As long as prgm present inside of "with open(---) as" indentation then filename is actively available and once prgm comes of "with open(---) as"

Indentation the file name closed automatically and this facility is called "Auto Closeability" of file.

• No need to close the file by using close() manually

program

program for demonstrating opening the file

ex① try:
 kvr=open("stud.data")
 except FileNotFoundError:
 print("File does not exist")
 else:
 print("\n Type of kvr=", type(kvr))
 print("File opened in read mode successfully")

ex② fp=open("stud.data", "w")
 print("\n Type of fp = ", type(fp))
 print("file opened in write mode successfully")

ex③ try:
 with open("stud1.data") as fp:
 print("file opened in Read mode")
 print("Type of fp =", type(fp))
 except FileNotFoundError:
 print("File does not exist")

ex④ with open("stud1.data", "w") as fp:
print("file opened in write mode")
print("Type of fp=", type(fp))

ex⑤ with open("stud2.data", "w") as fp:
print(" — — — — — ")
print("file opened in write mode")
print(" — — — — — ")
print("file name: ", fp.name)
print("file opening mode: ", fp.mode)
print("Is this file readable: ", fp.readable())
print("Is this file writable: ", fp.writable())
print("Is this file closed: ", fp.closed)
print(" — — — — — ")
print(" — — — — — out of with open() as — — — — — ")
print("Is this file closed: ", fp.closed)

ex⑥ same as ex⑤ try: ke and — .
isme (w) nahilikha hai except FileNotFoundError:
print("file does not exist")

ex⑦ same as ex⑤ but, ("a+") likhna hai.

ex⑧ same as ⑤, but, "x".
except FileExistsError
print("file already exist")

ex⑨ same as ⑤ but, ("x+")
except FileExistsError:
print("file already exist")

* writing the data to the File:-

→ To write the data to the file, we have two pre-defined functions present in file pointer object.

They are:

- 1) write()
- 2) writelines()

1) write()

⇒ syntax : file pointer obj. write (str data),

- This function is used for writing any type of data to the file in the form of str.

2) writelines()

⇒ syntax : file pointer obj. writelines (iterable object data),

- This function is used for writing any type of iterable object data to the file in the form of str.

Note:) write() and writelines() writes the data to file in the form of str by value by value but not all at once. it is one of the time consuming process.

program writing the data to the file.

ex① . with open("vs.data", "w") as fp:

fp.write(str(1000))

fp.write("vivek")

fp.write(str(34.11))

print("Data written to the file")

ex2. with open ("c:\\HYD\\addr1.data", "a") as fp:
fp.write("Guido van rossum\n")
fp.write("FNO: 3-4, Teilsider\n")
fp.write("python software foundation\n")
fp.write("metmelands-58\n")
print("Data written to the file")

ex3
X = {10: "Python", 20: "Java", 30: "C", 40: "DS"}
with open("authors.data", "a") as fp:
fp.writelines(str(x) + "\n")
print("Data written to the file")

Day 60:-

* Reading the data from the Files :-

→ To read the data from the file, we have two pre-defined functions present in file pointer object.

They are:

- 1) read()
- 2) readlines()

1) read()

⇒ syntax:

varname = filepointer.read()

- This function is used for reading entire data of the file and placed in LHS varname and its type is <class, 'str'>

2) readlines()

⇒ syntax:

varname = filepointer.readlines()

- This function is used for reading entire data of the file and placed in LHS varname and its type is <class, 'list'>.

ex① with open ("addr1-data", "r") as fp:

```
filedata = fp.read()
print(" — — — ")
print(filedata)
print(" — — — — ")
```

ex② with open ("addr2-data", "r") as fp:

```
filedata = fp.readlines()
print(" — — — — ")
X print(type(filedata))
for fd in filedata:
    print(fd, end=" ")
print(" — — — — ")
```

Q. write a python program which will read the data dynamically from the keyboard and write to the file.

→ with open ("Hxd.info", "a") as fp:

```
print("Enter ur data from key board press @ to stop:")
```

```
while (True):
```

```
    kbddata = input()
```

```
    if (kbddata != "@"):
```

```
        fp.write(kbddata + "\n")
```

```
    else:
```

```
        break
```

```
print("Data written to the file")
```

Q. write a python program which will copy the content of one file into another file.

→ srcfile = input("Enter source file:")

try:

```
    with open(srcfile, "r") as rp:
```

```
        dstfile = input("Enter Destination File:")
```

```
        with open(dstfile, "a") as wp:
```

```
            srcdata = rp.read() # reading the data from src file
```

```
            wp.write(srcdata) # writing the src file data
```

```
            print("File copied verify") # to Dest file
```

except file not found error:

```
    print("source file does not exist")
```